

דו"ח תיעוד פרויקט מסכם קורס "מבנה מחשבים ספרתיים" 361-1-4191

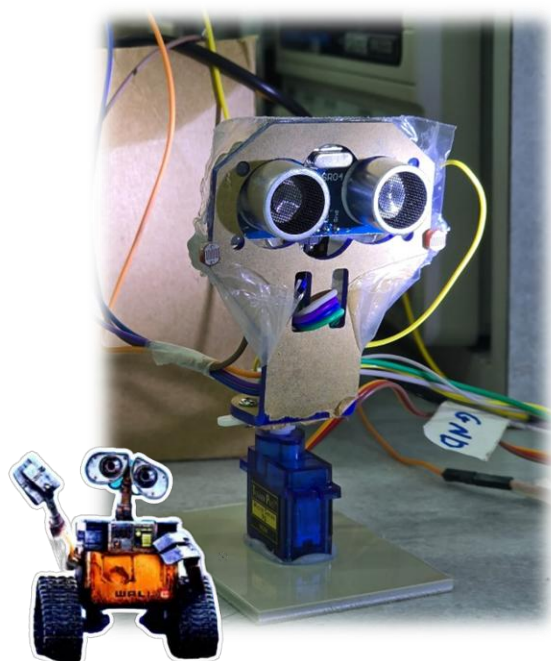
Light source and object proximity
detector system

מגישים:

לירון עדי 322528787

דביר סעד 214078792

01.09.2025



מטרת הפרויקט

מטרת הפרויקט היא לתכנן ולממש מערכת משובצת מחשב (Embedded System) מבוססת MCU לגילוי מקורות אור ולניטור אובייקטים במרחב.

- המערכת מבצעת סריקה בזווית של 180° באמצעות מנוע, Servo מדידת מרחק באמצעות חיישן **Ultra-Sonic** בטווח של 2–450 ס"מ כאשר ניתן להחליט על המרחק המקסימלי לזיהוי אובייקט להחלטתו.
- בנוסף יכולת זיהוי עצמת אור בעזרת חיישני LDR.

בקרת התנועה הזוויתית של המנוע מתבצעת באמצעות אות, PWM בעוד תהליך איסוף הנתונים מבוצע תחת גרעין הפעלה מבוסס FSM בשכבות אבסטרקציה ברורות, (BSP, HAL, API, APP)

בצד המחשב נבנה ממשק משתמש גרפי (GUI) בעזרת python אשר מסייעת לתקשורת בין ה-MCU לבין ה-PC ומתבצעת בתצורת RS-232 אסינכרונית, בקצב נתונים של 9600bps אשר הוגדר מראש

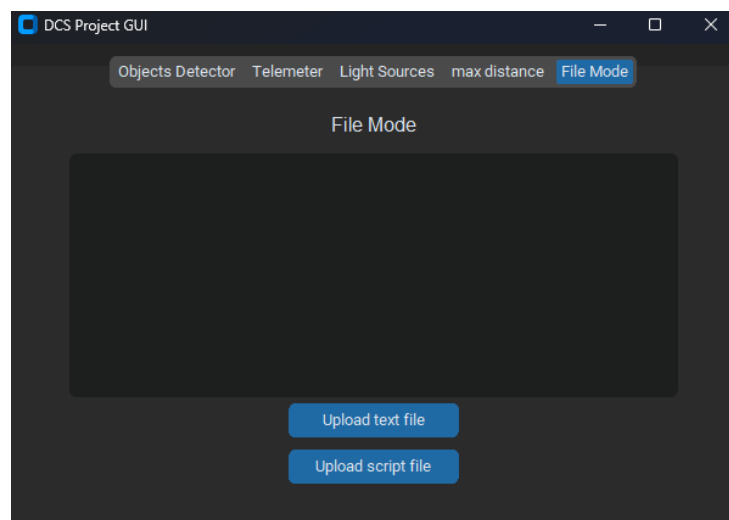
הפרויקט משלב ידע רב שנלמד במהלך הקורס ובמהלך הפיתוח, כולל עבודה עם: טיימרים, פסיקות, תקשורת טורית, בקרה על מנועי, Servo מימוש חיישני מרחק ואור, ותכנון ממשק משתמש.

המערכת מהווה בסיס למימוש יישומי "מערכת רדאר" קטנה לציווד קצה חומרת, תוך הקפדה על יציבות, דיוק, ועמידה במגבלות הנדסיות של מערכת משובצת מחשב בזמן אמת.

צד המחשב :

בצד המחשב בנינו GUI אשר עוטף קוד של Python שמבוסס על FSM שתפקידו הוא שליחת מספר המשימה הנדרשת, ובהתאם למספר המשימה הינו מקבל /שולח מידע נוסף אל/מהבקר.

תפריט הGUI:



תפריט זה מבוסס Tab כאשר אפשר לעבור בין כל Tab כרצוננו גם ללא הפעלת המצב ,
כאשר מצב ברירת התפריט היא פתיחת Tab שמורה על מטלה 1,

מצבי המערכת:

כעת נציג כל מצב שהמערכת מסוגלת לתמוך, ונראה את תפקידו בצד המחשב ובצד הבקר בנוסף נציג גם כיצד ביצענו את המטלה בפן החומרתי, ואת המגבלות שהיו לנו בזמן הפעולה.

מצב 1: ניטור אובייקטים במרחב

בשגרה זו, התבקשנו לממש מערכת אשר מסוגלת לזהות אובייקטים (מרחק, זווית ורוחב) באופן דינאמי במרחב.

כאשר המרחק המקסימלי לזיהוי אובייקט גם נתון להחלטתנו על ידי מערכת הGUI, ואילו טווח הזיהוי הינו בין 0 ל 180 מעלות.

צד הבקר

לצורך ביצוע המטלה היה צורך לשימוש ברכיב UltraSonic לצורך זיהוי האובייקטים, ומנוע מסוג Servo בכדי לבצע זיהוי בטווח הרצוי (180 מעלות).

לצורך הפעלת הרכיבים השתמשנו בTimerA0 וTimerA1 כאשר הפעולות שהיו הם הפעלת אות PWM לצורך הזזת המנוע, שינוי ערך freq של האות הPWM בכדי "לשלוט" על הזווית

ושימוש במצב מיוחד של השעון input-Capture בכדי להשתמש ב UltraSonic לצורך חישוב הפרש הזמנים בין שליחה קול לקבלת הקול חזרה.

ולכן החלטנו לבצע כך:

- TimerA0- מטרתו הייתה ליצור גל הPWM
- TimerA1- היו לו 2 מטלות, מטלה ראשונה הזזת המנוע ב 3 מעלות כלומר שינוי התדר של PWM בעזרת העלאת פסיקה במצב של Compare Mode .

לאחר כיבוי מצב CM עברנו אל מצב Input Capture בכדי לקלוט את המידע שמגיע מתוך UltraSonic.

בכך שבחרנו שיש שימוש רק של מצב אחד בכל פעם אנו מנענו מקרים של התנגשות בין שני המצבים מכיוון ששני המצבים משתמשים באותו מונה.

צד המחשב

תחילה הGUI שולח אות אל הבקר אשר מחליט על מצב משימה 1 , ולאחר מכן בחלון הכתיבה מתקבלות 180 הדגימות בצורה של 1 byte ששולח את מספר הזווית ולאחר מכן 2-byte שנשלחים ובהם המרחק של הרכיב .

לאחר מכן אנו מבצעים בעזרת python ניתוח של הנתונים שהתקבלו כדי "לזהות" אובייקט ולאחר מכן סידור הנתונים והדפסתם אל הGUI .

חישוב נצילות :

עבור כל שליחת byte של מידע מתקיים כי :

נשלח 1 stop bit

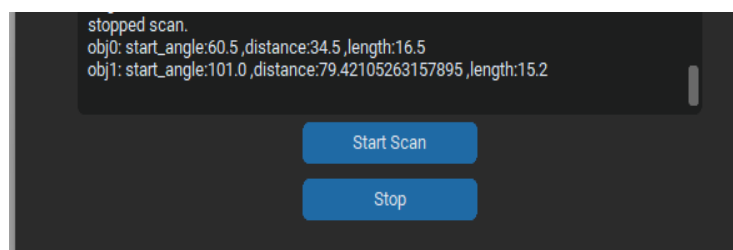
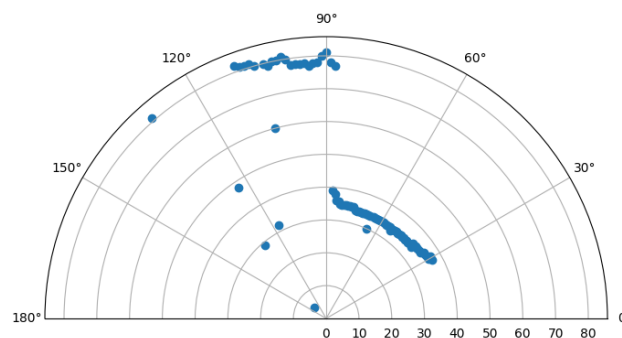
נשלח 1 partiy bit

נשלח 8 bit של מידע

ולכן בפועל רק 8/10 מתוך כל המידע שנשלח הוא אכן מידע שאנו משתמשים בו ולכן :

הנצילות הינה 80%

להלן דוגמא לניתוח התוצאות בצד במחשב :



כפי שניתן לראות הבקר סרק את שטח ב180 מעלות והראה לנו את תוצאות 180 הדגימות , כמובן שסיננו מראש דגימות אשר אינם בטווח שהוגדר מראש להיות המרווח המקסימלי לזיהוי .

אני מזיהים 2 גושים של דגימות צפופים וננחש שאלו היו האובייקטים שלנו כאשר יש לנו פונק שתיקח דגימות אלו ותמיר אותם לאובייקטים ותדפיס את ערכם אל הGUI כפי שניתן לראות ליעל .

מצב 2: Telemeter

במצב זה בעזרת הGUI אנו שולחים זווית אל הבקר ומזיזים את המנוע והUltraSonic אל הזווית הנ"ל.

ולאחר מכן אנו מראים בצורה דינאמית את המרחק שהגלאי מזהה בזווית הנתונה.

לצורך כך השתמשנו בהגדרות אשר הוגדרו למטלה 1 למעט הזזת המנוע מכיוון שבמצב זה המנוע זז בצורה חד פעמית כאשר אנחנו שולחים זווית חדשה.

צד הבקר

כפי שמוזכר ליעל, השימוש בTimerA1 ובTimerA0 זהים כמעט לחלוטין לתפקידם במטלה 1 למעט הזזת תדר של TimerA0.

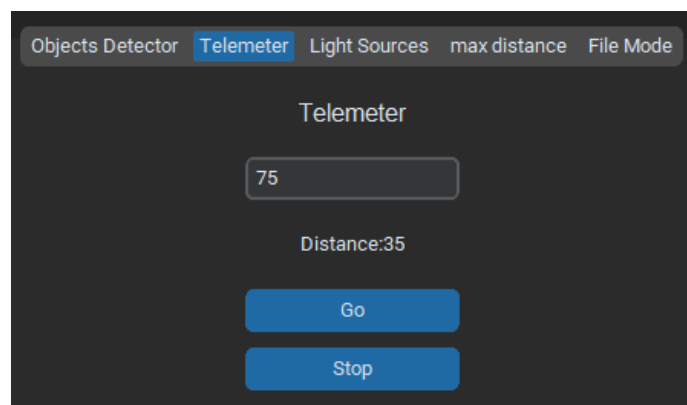
צד המחשב

לאחר שליחת מספר המטלה אל מחשב, מערכת הGUI מבקשת שליחה של מספר בין 0 ל 180 בכדי להורות למנוע לזוז אל הזוויות ולהתחיל במדידה.

לאחר מכן הpython מחכה לקבלת המידע באופן רציף והצגתו בGUI.

לאחר מכן נתקלנו בבעיה, לא ידענו כיצד לדעת מתי להפסיק את מדידת הנתונים בכדי שנוכל לצאת מהמצב ולעבור למצב אחר בכדי לא לתקוע את ריצת התוכנית ולגרום לקריסתה.

וכתוצאה מכך החלטנו להוסיף כפתור מיוחד (stop) אשר תפקידו הוא עצירה המערכת וכניסה למצב שינה בצד הבקר.



מכיוון שמצב זה משתנה דינמית ואין לו הגדרה מוגדרת לסיום הפעולה , הוספנו כפתור אשר פעולתו היא עצירה המצב וכניסה למצב שינה.

מצב 3: מערכת לזיהוי מקורות אור

במצב זה , התבקשנו לממש מערכת לניטור מקורות אור באופן דינאמי במרחב , במרחק מוגדר דרך ממשק המשתמש לביצוע סריקה יחידה , בטווח שבין 0 ל180 מעלות.

לצורך המטלה היינו צריכים להשתמש במספר רכיבים , חיצוניים ופנימיים .

ראשית היה שימוש ב2 רכיבי LDR אשר הינם נגדים שמסוגלים להראות שינוי במתח כתוצאה מהארה .

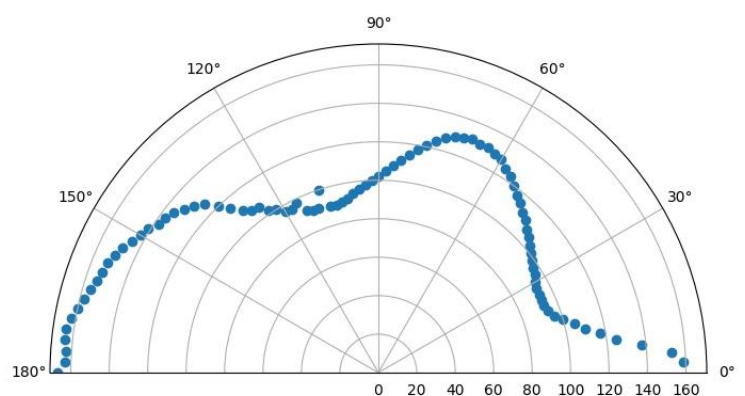
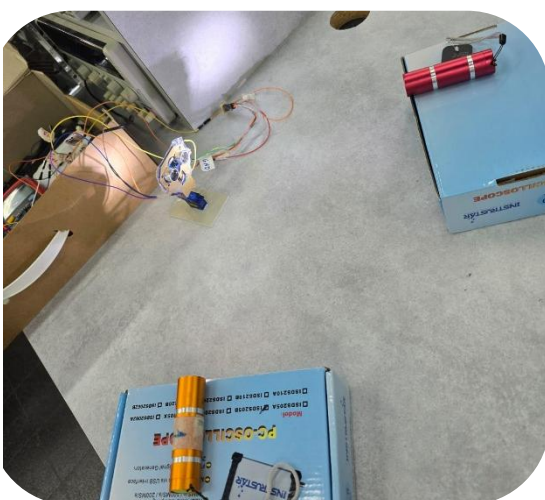
ונשתמש ברכיב פנימי של הבקר , רכיב ADC10 אשר מבצע המרה של ערך אנלוגי (מתח) אל ערך דיגיטלי כאשר במקרה שלנו מדובר מספר בינארי בין 10 ספרות כלומר מספר בין 0-1023 ומתאים כל ערך מתח לערך ספרתי.

על מנת להתאים כל מדידה לתוצאות נצטרך להיעזר בכיול אשר נבצע בתחילת ריצת התוכנית .

פונק הכיוון מוגדרת להיות מדידת 10 דגימות של אור זווית של 90 מעלות במרחקים משתנים , בכדי לקבל אינדיקציה על התוצאות של האור .

מכיוון שהאור וערכי המתח יכולים להשתנות בהתאם לסביבה בה אתה נמצא .

בעזרת הGUI ביצעו שרק לאחר מטלת הכיול נוכל לבצע את המשימה הנ"ל .



בתמונה הימנית ניתן לראות את תוצאות הסריקה של שני אובייקטים של אור מהתמונה השמאלית.

אפיון המערכת

א. סעיף גילוי אובייקטים במרחב

זמן מחזור האיסוף של הדגימות של סריקה בודדת חושב כך:

$$T = NumSample * SMCLK * Sample_{Time} * Rotation_{Time}$$

כאשר אנחנו בחרנו :

$$1\mu s - T_{smclk}$$

מספר הדגימות נבחר כתוצאה משיקול הנדסי של בחירת ערכים יותר יציבים של דגימות מאשר מגוון של דגימות ולכן מספר הדגימות הינו 180, כלומר הזזה של 1 מעלות כל הזזה.

זמן דגימה - זמן הדגימה נבחר מתוך פיזי של רכיב Ultra-Sonic שהינו יכול למדוד רכיב עד 4 מטר ולכן נדרש למצוא את הזמן שלוקח לדגום אובייקט במרחק זה תוך התחשבות מהירות הקול לכן נבחר : $T_{sample}=62msec$.

זמן הסיבוב נבחר להיות 0xFFFF.

ולכן סה"כ זמן המחזור של דגימה הינו :

$$(0.062 * 0.0655) \times 180 \approx 0.1275 * 180 \approx 22.95 \text{ sec}$$

שליחת המידע

המידע נשלח באמצעות היכולות של UART בשיטת Threading שבה אנו שולחים חבילות של Byte אך לא עוצרים לגמרי את המעבד בניגוד לשליחה של Block .

המידע נשלח כחבילה דחוסה של 3-Byte כאשר החלוקה נעשתה כך:

1-Byte שומש עבור שליחת הזווית. (ייצוג מספר בין 0 ל 180)

2-3 Byte שומש עבור שליחת המרחק בסנטימטרים (ייצוג מספר בין 0 ל 400) כאשר המספר התחלק ל MSB ול-LSB בכדי לשלוח חבילה.

עיבוד המידע

את המידע היה נדרש לעבד רמות עד להצגתו במחשב, נסביר את צד הבקר בחישוב ואת צד המחשב של החישובים.

בצד הבקר מתקבלים הקלט של מונה השעון בזמן שליחת גל הקול (trigger) ומונה השעון זמן קבלת הגל הקול חזרה מהאובייקט (echo), לאחר מכן נדרש לנו לחסר בין המספרים ולקבל את הזמן הכולל (במחזורי smclk) שלקח לגל הקול ללכת ולחזור.

לאחר מכן היינו צריכים לנרמל בתדר השעון ולבצע חישוב של מציאת המרחק מתוך הכפלה של הזמן ושל המהירות.

לאחר מכן נדרש שלחנו **אל המחשב** חבילה אשר מרכיבה דגימות של זווית ושל מרחק בסנטימטרים.

תיאור הפונקציה :

בכדי לזהות אובייקטים מתוך הדגימות בנינו פונקציה אשר מאתרת מספר רציף של דגימות בעל ערך מרחק זהה ומגדירה אותם כאובייקט, תוך חישוב של האורך של האובייקט בעזרת גאומטריה של אורך קשת כלומר: $length = 2 * distance * \sin((\theta_{finish} - \theta_{start})/2)$

-במהלך בניית הפונקציה נתקלו בכמה בעיות שצצו עקב חומרה.

גילינו כי לפעמים אחת למס בדיקות מתקבלת תוצאה לא הגיונית של שסוטה מהבדיקות האחרות (ככל הנראה מאקו של החדר או בעיה חשמלית) ולכן כדי להתגבר על בעיה זו, הוספנו דגל שמתעלם בקפיצות יחידות ולא רצוניות בבדיקות.

ב. סעיף גילוי מקורות אור במרחב

מציאת זמן מחזור של הדגימות של מקור האור כמו בסעיף הקודם:

$$T = NumSample * SMCLK * Sample_{Time} * Rotation_{Time}$$

כאשר אנחנו בחרנו :

$$1\mu s - T_{smclk}$$

מספר הדגימות נבחר כתוצאה משיקול הנדסי של בחירת ערכים יותר יציבים של דגימות מאשר מגוון של דגימות ולכן מספר הדגימות הינו 90, כלומר הזזה של 2 מעלות כל הזזה.

זמן דגימה -זמן הדגימה נבחר מתוך פיזי של רכיב ADC10 שדוגם את הערך של האור, זמן הדגימה של רכיב זה הוא $smclk * 13$

זמן הסיבוב נבחר להיות 0xFFFF.



ולכן סה"כ זמן המחזור של דגימה הינו :

$$(0.000013 + 0.0655) \times 90 \approx 90 * 0.656 \approx 40 \text{ sec}$$

שליחת המידע

מכיוון שכל רכיב LDR משמש רק בחצי מהסיבוב , נשלח בכל פעם רק את הדגימה הרלוונטית ולכן כדי לחסוך במקום שלחנו סהכ 2 בתים.

1 Byte לצורך הזווית בין 0 ל 180.

הבית השני נבחר תוך התחשבות וחסכון במקום , לקחנו את הדגימה שהיא מיוצגת כ-10 bits. ועשינו חיתוך של 2 סיביות ה-LSB- תוך התחשבות בכך שסטייה של 2 LSB לא יובילו לשינוי משמעותי .

זיהוי המקורות:

לאחר ביצוע הקליברציה ושליחת המידע (הקליברציה) אל המחשב ושליחת הדגימות בזמן אמת, הפונקציה מבצעת :

הגדרת *threshold* כלומר סף מינימלי שעבורו נגדיר שהתחלנו לזהות אלומה של אור כלשהי, לאחר כניסה לאלומה אנו אוגרים את המידע עד שאנחנו יוצאים חזרה מהאלומה.

לאחר שנכנסנו ויצאנו מאלומה של אור נחפש מה הערך המינימלי במערך

(מכיוון שככל שהערך יותר נמוך ככה אנחנו קרובים יותר לאור) ונחפש איפה הערך הזה נמצא בערכי הקליברציה שלנו .

האינדקס הקרוב ביותר לערך הנמדד יוגדר להיות המרחק של האור מהגלאי .

ג. סעיף מד מרחק

כפי שצוין בסעיף א , הערך מהאובייקט הוגדר ע"י חישוב שהתבסס על הזמן

(ביחידות של smclk) שלקח ultra-Sonic לשלוח את גל הקול ולקבל (echo) בחזרה .

לאחר מכן הוא עובר חישוב של נרמול ב smclk והכפלה במהירות כדי לקבל יחידות של אורך.

ולאחר מכן אנו שולחים את המידע בצורה דחוסה של 2 byte של int שמייצג את המרחק בסנטימטרים.

תדר הדגימות

מכיוון שמתבצעת הזזה יחידה, זמן מחזור הדגימות תלוי רק ברכיב Ultra-Sonic ולכן זמן המחזור הינו: $T=62.5\text{ms}$

החלטה על שינוי מרחק:

מכיוון שאנו רוצים להראות בצורה דינאמית צריך לבדוק כל זמן מסוים האם יש שינוי במרחק ולכן החלטנו שעבור כל זמן מסוים 50ms אנחנו נשלח נשוב בקשה למדידת מרחק ונדפיס את המרחק המתקבל.

זוהי בכדי לחסוך בזמן ובהספק שנובע מתוך שליחה ממשוכת ולפעמים מיותרת של עוד מידע.

ד. מערכת קבצים

ואילו הקבצים מאוחסנים בטווחים 0xF600-0xFDFF אשר הם מתפרשים על 4 סגמנטים כלומר סגמנטים 1-4, גודל כל סגמנט הינו 512B. לעומת זאת מבני הנתונים (ה - struct) שאחראיים על ניהול מערכת הקבצים מאוחסנים ב-flash information memory ושם הם מאוחסנים ב-B segment עד D כאשר גודל כל סגמנט באזור זה הינו 64B.

נסביר בקצרה כיצד עובדת מערכת הקבצים אצלינו + נצרף פסאודו קוד של הפונק המערכת:

מיפוי זיכרון:

- **Data Area (2 KB):** אזור רציף בפלאש לקבצי הנתונים עצמם.
 - **Directory (~256 B):** בטבלת רשומות קבועות-גודל.
- אין "עדכונים במקום" בפלאש → רשומות נכתבות בגישת append ומסומנות למחיקה ע"י הורדת ביט.

File Mode – Pseudo-code Summary

אתחול ופורמט בזינק

```
main():
    state = idle
    sysConfig()
    erase_all_file_segments()
    lcd_init(); TIMERS(); ADC(); UART()
    loop:
        switch(state):
            case FILE_MODE:
                if run_script_request: run_script_mode(entry_ptr)
                if file_upload_finished:
                    write_struct_to_flash(&my_script, struct_ptr)
                    struct_addresses[num_files++] = struct_ptr
                    struct_ptr += sizeof(FileEntry)
                    display_scrolling_view()
```



```
file_upload_finished = 0
```

UART – ISR קליטת קובץ דרך

```
USC10RX_ISR():  
if byte=='@': mark start  
else if last=='@': choice = byte; reset parsers  
  
switch(choice):  
case 'D':          // File Mode  
    ++count_file  
    if count_file==1: my_script.type = (rx=='T'?TEXT:SCRIPT)  
    else if 3<=count_file<=5: file_size[idx++] = rx  
    else if 6<=count_file<=17:  
        my_script.size = array_to_num(file_size)  
        my_script.name[name_idx++] = rx  
        my_script.flash_address = script_pointer  
    else if count_file>=18:  
        if written < my_script.size:  
            write_byte_to_flash(script_pointer, rx)  
            ++script_pointer; ++written  
        if written == my_script.size:  
            file_upload_finished = 1
```

(Directory) יצירת רשומת קובץ

```
write_struct_to_flash(entry, dest_flash):  
for i in 0..sizeof(FileEntry)-1:  
    write_byte_to_flash(dest_flash+i, ((char*)entry)[i])
```

הפעלת מצב script ותפריט משתמש

```
display_scrolling_view():  
line1 <- name at struct_addresses[top_index]  
line2 <- name at struct_addresses[top_index+1]  
  
lcd_show_page(file_struct_ptr):  
content_ptr = find_content_pointer(file_struct_ptr)  
show_text(content_ptr)  
  
run_script_mode(file_struct_ptr):  
size = find_size(file_struct_ptr)  
content = find_content_pointer(file_struct_ptr)  
for each line in content:  
    script_command_machine(line)
```

ה. כיוול חומרה

לאחר בדיקה ערכי הTon שלנו הינם:

עבור 0 מעלות: $TA0CCR1 = 420$

עבור 180 מעלות: $TA0CCR1 = 2040$

בעבור המערכת שלנו:

TimerA0 – מכוון למצב output_mode7 והוא האחראי על שליחת אות PWM אל המנוע,

TimerA1- יש 2 תפקידים תפקיד ראשון הינו מצב של MC_1 בכדי לקבוע כל כמה זמן נבצע הזזה של המנוע כלומר: מוסיף x אל רגיסטר TA0CCR1 ובכך בעצם מזיז את המנוע במעלה.

תפקיד שני של הטיימר הינו CM_3 כלומר אנחנו מבצעים לכידה של ערך הטיימר בזמן קבלת עלייה וגם קבלת ירידה, וזאת לצורך מדידת המרחק בעזרת רכיב ultra-Sonic.

אות הPWM פועל בכל זמן שTimerA1 נמצא על מצב של עליה כלומר רק בזמן הזזת המנועה הוא פועל, וזאת בכדי שבזמן המדידה של הערכי המרחק לא ייוצר תזוזה של הבקר שיגרם לסטיות ולמדידות לא חלקות.

ו. חישן המרחק

עבור חישן המרחק היינו צריכים להתחשב בשיקול הנדסי של זמן שליחת הפולס של ה trigger בכדי לא להגיע למצב שאנו שולחים עוד פולס לפני הפולס קודם חזר ולכן: הזמן שלוקח Ultra-Sonic לזהות רכיב שנמצא במרחק 4 מטר הוא לכל היותר 60ms ולכן בחרנו שתדר pulsen יהיה 62.5ms ואילו התדר נבחר: $TA1CCR1 = 0x0020$ 10us

כאשר בפועל הרכיב מרכיב לנו Pulse ריבועי שמשתנה בזמן הטריגר ומשתנה שוב בזמן שהecho חוזר, וכך בעצם יש לנו את הפרש הזמנים בעזרת Input-capture ובעזרתו אפשר לחשב את המרחק של האובייקט.

ז. חישן מרחק ממקור אור

השלמת הערכים של 50 הדגימות מתוך 10 הדגימות נעזרנו בתכונת הלינאריות של הדגימות.

ולכן החלטנו להשלים לינארית את הדגימות, לצורך כך לכל 2 דגימות צמודות שחישבנו שיפוע ונקודת התחלה והשלמנו את הדגימות בעזרת לולאת for פשוטה.

לאחר מכן שלחנו את 2 יחידות הכיוול לflash ואל הPC.

עיבוד המידע

בצד המחשב לקחנו את הדגימות והחלטנו לבצע מספר שינויים: ראשית, בזמן הסריקה נבחר רק LDR אחד שיימדד כאשר הזווית קטנה מ90 מעלות נבחר לנתח את הLDR השמאלי, ולהפך.

בנוסף יש שוני מאשר ניתוח המרחק של מטלה 1, מכיוון שיש להתחשב בתוכנה של האור.
האור מתפשט לכן כאשר אנחנו נרצה למצוא את מקור האור, אנחנו נצפה לקבל תוצאות בצורה של פעמון הפוך, כלומר ככל שנתקרב למקור האור תוצאות ה ADC יהיו קטנות יותר ולכן בתוך כל פעמון כזה נרצה למצוא את ערך ה Min
לאחר שמצאנו את הערך המינימאלי נרצה לזהות את המרחק של המקור, לצורך כך נעזר בכיול שלנו.
נעבור עם הערך שמצאנו על המערך באורך 50 ונחפש את האינדקס שעבורו התוצאה הכי קרובה.
ונגדיר את האינדקס להיות המרחק.

ח. מבנה נתונים ואלגוריתמים

קליטת הפריים (RX) מתבצעת באמצעות פסיקת UART RX (ISR_UART_RX).
הנתונים הנכנסים נשמרים במבני נתונים זמניים ומחולקים לשדות:

- **Opcode** - שדה באורך קבוע של 2 תווים.
- **Operand** - שדה באורך משתנה בהתאם לפקודה.

המימוש עושה שימוש במבנה FIFO (circular buffer) לצורך קליטה רציפה של מידע.

גודל הפריים ב־ RX הוא משתנה.
שדה ה־ Opcode תמיד קבוע באורך של 2 תווים, ואילו שדה ה־ Operand יכול להשתנות לפי סוג הפקודה.
לכן, אורך הפריים הכולל איננו קבוע.

מבנה נתונים של פריים TX

פריים השידור (TX) נבנה על ידי ה־ MCU בהתאם למצב המערכת:

- Type (בית אחד: לדוגמה 'S')
- Angle (בית אחד המייצג זווית)
- Distance (שני בתים LSB + MSB)

במצבים מסוימים משודרים רק נתוני מרחק.

האם גודל פריים TX קבוע או משתנה?

גם גודל פריים ה־ TX הוא משתנה.

- במצב סריקה – הפריים כולל 3 בתים. (Angle, Distance_LSB, Distance_MSB)
- במצב מדידה בודדת – ייתכן וישודרו רק 2 בתים של מרחק.

לפיכך, הגודל תלוי בהקשר.

תמיכה בגילוי שגיאות

המערכת תומכת בגילוי שגיאות באמצעות מנגנון Checksum 256:

- בצד הבקר: מתבצע חישוב סכום של כל בתי הפריים מודולו 256, והתוצאה מצורפת כבית נוסף בסוף הפריים.

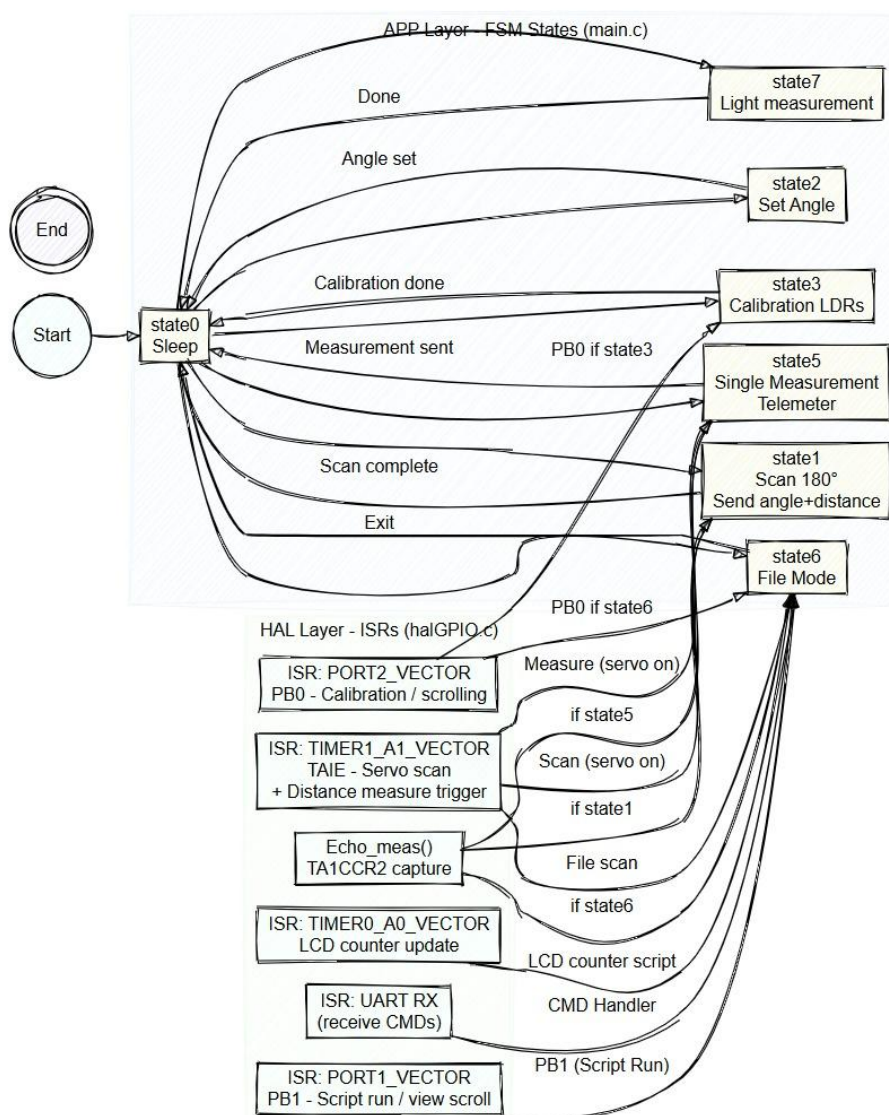
- בצד המחשב : מתבצע חישוב חוזר והשוואה. אם הערכים תואמים – הפריים תקין; אחרת – הפריים נפסל ונשלחת בקשה לשידור חוזר.

פרוטוקול Stop-and-Wait ARQ

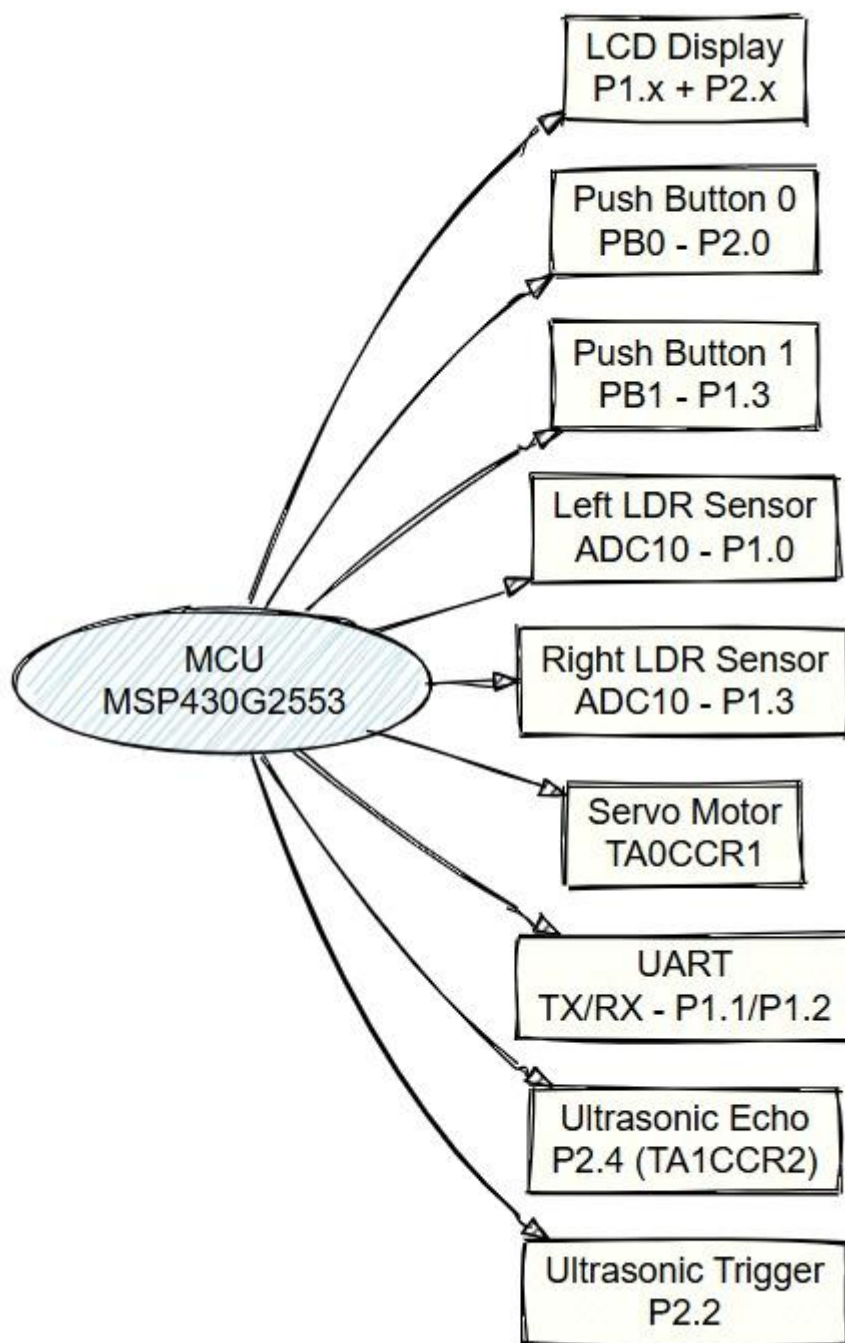
המערכת תומכת בפרוטוקול Stop-and-Wait ARQ:

- הבקר שולח פריים וממתין ל-ACK מהמחשב.
- אם מתקבל ACK בזמן – המערכת ממשיכה.
- בצד המחשב: נשלח ACK אם הפריים תקין.
- מנגנון זה מבטיח תקשורת אמינה ותקינה בערוץ UART.

:FSM



שרטוט חומרת קצה:



הסבר קצר על כל פונק בצד המחשב:

טאב 1 – Objects Detector

- **task1_start_scanning()** מתחיל סריקת אובייקטים: שולח פקודת התחלה ל-MCU ומפעיל את הלולאה.
- **objects_scan** לולאת קריאת תוצאות סריקה: קוראת 3 בתים (זווית + מרחק), מציגה וממשיכה כל ms.50.
- **stop_scan()** עוצר את הסריקה, שולח פקודת עצירה, מזהה אובייקטים מתוך המדידות ומצייר גרפים.
- **identify_objects(my_list)** אלגוריתם זיהוי אובייקטים: מוצא רצף מדידות קרוב, מחשב מרכז זווית, מרחק ואורך.
- **plot_scan_polar(measurements, title)** מצייר תרשים פולרי של המדידות (זווית מול מרחק).

טאב 2 – Telemeter

- **task2_scanning()** מצב מדידת מרחק: שולח פקודת מדידה, קורא 2 בתים (מרחק) ומעדכן GUI.
- **task2_stop_scanning()** עוצר את מצב המדידה ושולח פקודת עצירה.
- **send_angle()** שולח זווית נבחרת ל-MCU למדידה בודדת, מפעיל task2_scanning לקריאת התוצאה.

טאב 3 – Light Sources

- **calibration()** מתחיל כיול חיישני אור, שולח פקודת 'C' ל-MCU וממכה לסיום.
- **check_if_finish()** בודק אם הכיול הסתיים, ואם כן טוען נתוני הכיול ומציג כפתורי סריקה.
- **show_light_scan_controls()** מציג פקדים לסריקת אור לאחר כיול.
- **start_light_scan()** מתחיל סריקת מקורות אור ושולח פקודת 'L' ל-MCU.
- **light_scan()** לולאת סריקה: קוראת זווית + עוצמת אור, מציגה, ובסיום מחשבת מקורות אור.
- **light_analysis(left_calib, right_calib, measurements_light)** מזהה מקורות אור על סמך הכיול ומדידות בפועל.
- **find_nearest_distance(calib_values, measured_value)** מוצא מרחק מקורב לפי ערך הכיול הקרוב ביותר.

- **get_calibration()** – קורא 100 בתים מה- UART: 50 שמאל, 50 ימין, ושומר כתוצאות כיוול.

טאב 5 File Mode –

- **text_file()** – מעלה קובץ טקסט למיקרו: שולח מטא-דאטה ותוכן הקובץ.
- **script_file()** – מעלה קובץ סקריפט: מתרגם פקודות לקוד hex, שולח ל- MCU ומתחיל קריאה חזרה.
- **choose_file()** – מאפשר לבחור קובץ, מתרגם/שולח אותו למיקרו ומציג את המידע ב-GUI.
- **translate_script_file(lines)** – מתרגם פקודות סקריפט לפי טבלת opcode לערכי hex.
- **start_uart_handler()** – מאתחל קריאת UART, מאפס מונים ומתחיל לולאת האזנה.
- **wait_for_uart_response()** – מחכה לקריאה מ- UART: מזהה הודעות לפי opcode (מידת מרחק או סריקה), מציג ומפסיק בסיום.

הסבר קצר על כל פונק בצד הבקר בשכבת הAPI

read_struct_from_flash קוראת מבנה מה- Flash לתוך RAM ע"י העתקת בתים אחד-אחד.

init_count_up מאתחלת מונה ל-000 על התצוגה.
inc_lcd מעלה את הערך על ה- LCD כל טיק עד שמגיע לערך יעד.
init_count_down מאתחלת מונה עם ערך התחלתי מה- (operand על ה- LCD מורידה את הערך על ה- LCD כל טיק עד 000.
print_operand_char מחזירה את התו הראשון המשמעותי מתוך המחרוזת מדלגת על 0, רווח. (\n),

rra_lcd מציבה תו אחד על המסך בכל קריאה, מתקדם תא-תא (שורה 1 ואז 2).
set_delay מגדירה דיליי בטיימר (A0 לפי operand).
read_from_flash מחזירה בית מה- Flash בכתובת נתונה.
write_struct_to_flash כותבת מבנה שלם לזיכרון Flash.
write_byte_to_flash כותבת בית יחיד ל- Flash כולל ניהול erase ו- (unlock).
get_flash_segment_start מחשבת את תחילת הסגמנט של כתובת נתונה (Main/Info).
erase_flash_segment_auto מוחקת אוטומטית סגמנט ב- Flash לפי הכתובת.

erase_flash_segment מוחקת סגמנט יחיד ב־Flash ע"י (dummy write).
erase_all_file_segments מוחקת כמה סגמנטים קבועים בזיכרון (לניהול קבצים).
print_struct_names_from_flash מדפיסה ל־LCD את שמות הקבצים ששמורים בזיכרון.
script_command_machine מפרקת פקודה, (opcode+operand) מריצה, וממתינה לסיום.
find_content_pointer מחזירה פוינטר לתוכן של קובץ) מה־struct בזיכרון.
find_type מחזירה את סוג הקובץ מה־struct בזיכרון.
find_size מחזירה את הגודל (size) של הקובץ מה־struct בזיכרון.
show_text מציג טקסט על ה־LCD שתי שורות, גולל שורות חדשות, שומר מיקום לקריאה הבאה).
lcd_show_page מציג עמוד טקסט ב־LCD מתוך תוכן הקובץ.
scroll_text_on_button_press מציג את המשך הטקסט בלחיצה (בהתאם למיקום שנשמר).
run_script_mode מריץ סקריפט מתוך Flash, שורה־שורה, ומבצע פקודות.
display_scrolling_view מציג 2 שמות קבצים (מהרשימה) על ה־LCD.
scroll_view_on_button_press מעביר את חלון ההצגה לשמות קבצים הבאים.
parse_command מפרקת מחרוזת פקודה ל־opcode ו־operand.
execute_command מבצע פעולה לפי ה־opcode מונה, טיימר, סרוו, מצב שינה וכו.
servo_deg - מסובב סרוו לזווית נתונה.
servo_scan - סורק עם הסרוו בין שתי זוויות (התחלה-סוף).
array_to_num ממיר מערך chars (בצורת ספרות) למספר שלם.
calibration לוקחת דגימות מה־LDRs ושומרת לטבלאות הכיול.
Fill_calib_Linearization - מבצעת לינאריזציה בין נקודות הכיול וממוצע בין LDRs.
Light_detec_left - קוראת ערך אנלוגי מהחיישן השמאלי.
Light_detec_right - קוראת ערך אנלוגי מהחיישן הימני.
calibration_to_pc - שולחת את מערך הכיול למחשב דרך UART.